

NileFlow — Team Execution Playbook

Internal working document: timeline, risk zones, fallback plans, and stack reference

1. The Pitch (memorize this)

"NileFlow tells you, in real time, whether your commute through Cairo or Alexandria is about to be miserable, and why. We stream live traffic, transit, and weather data through Kafka, process it with Spark to compute a live congestion index, detect anomalies, and push alerts to Discord while showing everything on a live map dashboard."

2. Problem and Solution (recap)

Problem: Greater Cairo's transit network (formal metro/bus + informal minibuses) has no real-time information layer. No public, data-driven view shows where the network is congested right now, and traffic congestion is one of the most cited daily-life pain points in Egypt — yet no open real-time analytics combine transit, traffic, and weather to explain how bad it is, where, and why.

Solution: A real-time congestion and transit-delay monitoring platform for Greater Cairo and Alexandria — ingest live traffic + transit + weather data, detect anomalies with Spark, store results across PostgreSQL / Cassandra / Elasticsearch, visualize on a live dashboard, and alert via Discord webhook when something goes wrong.

3. Timeline (Deadline: 15th of next month — approx. 4-5 weeks)

Week	Focus	Key Outputs
Week 1	Infra + reference data + start Spark streaming research/prototype in parallel	Docker Compose skeleton running (Kafka, Spark, Postgres, Cassandra, ES, Redis, Airflow); GTFS/route data loaded into Postgres; first Kafka producer (traffic) sending data
Week 2	All Kafka producers live + Spark streaming joins working	Weather + synthetic vehicle position producers running; Spark Structured Streaming consuming all 3 topics with windowed join (this is the core risk — start early)
Week 3	Congestion index + anomaly detection + storage writes	Rolling congestion index computed; anomaly detection logic flags severe events; Cassandra + Elasticsearch receiving data; Discord webhook sending test alerts
Week 4	Dashboard + Airflow DAGs + integration	Live dashboard (Streamlit/Grafana) showing map + time-series + alerts; 1-2 Airflow DAGs (daily refresh, data quality check); full pipeline running end-to-end
Week 5 (buffer)	Polish, debug, rehearse demo, documentation	README, architecture diagram, demo script, presentation slides; cut any unfinished stretch items per the fallback table below

Golden rule: start the Spark Structured Streaming work in Week 1, in parallel with infrastructure setup. It is the single biggest risk — give it the most time and your most experienced person.

4. Danger Zones (ranked by risk)

Rank	Risk Area	Why It's Dangerous	Mitigation
1	Spark Structured Streaming (multi-topic joins, windowing, watermarking)	Hardest concept on the stack. Teams commonly lose days to silent join failures, late-data issues, and "no output" bugs.	Assign your strongest person here. Start Week 1. Build a minimal single-topic job first, then add joins incrementally.
2	Docker Compose with 6+ services	Kafka, Spark, Postgres, Cassandra, Elasticsearch, Redis, Airflow together are resource-heavy; networking/port conflicts eat time without producing visible progress.	Build the skeleton in Week 1 before features are added. Bring services up one at a time, verify connectivity, then layer on.
3	Live traffic API quota (Google Routes API / OSRM)	Free-tier quota can run out mid-development or mid-demo if polling too frequently across many corridors.	Limit to 4-6 key corridors, poll every 2-5 min. Cache/log responses for demo replay if quota runs low. OSRM self-hosted as backup (adds setup time).
4	Transport for Cairo / GTFS data access	Lowest risk of the four, but possible 1-2 day detour if download requires approval or the format is messier than expected.	Start with a small hand-built corridor list (Ring Road, Corniche, October Bridge, Salah Salem, 26th of July) on Day 1. Swap in real GTFS later if easy to get.

5. Scope Tiers — What to Cut If Time Runs Short

Build in this order. Everything in Tier 1 is non-negotiable — it's the minimum that proves the concept end-to-end. Tier 2 makes it impressive. Tier 3 is polish for if you're ahead of schedule.

Tier 1 — Core (must finish)

- Kafka producers: traffic + weather (synthetic vehicle pings can come later)
- Spark Structured Streaming: single-topic processing first, then add the join
- Congestion index calculation + basic threshold-based anomaly flagging
- PostgreSQL (reference data) + Cassandra (metrics)
- Basic dashboard (even a simple Streamlit chart + table is acceptable)
- Docker Compose running the core services together

Tier 2 — Impressive (high value, do if Tier 1 is on track)

- Synthetic vehicle position producer + live map view on the dashboard
- Discord webhook alerts for severe congestion events

- 1-2 Airflow DAGs (daily refresh + data quality check)
- Weather correlation insight (rain vs. congestion spike) shown on dashboard

Tier 3 — Stretch / Polish (cut first if behind schedule)

- Elasticsearch for alert/event search (Cassandra alone can carry the demo)
- Real TfC GTFS data (fallback corridor list is acceptable for the demo)
- Historical trend analysis / multi-week baselines
- Grafana instead of Streamlit (only switch if Streamlit is already working and time allows)

6. Tech Stack Quick Reference

Component	Tool	Role
Streaming backbone	Apache Kafka	Carries traffic, weather, and vehicle position events between producers and Spark
Stream processing	Spark Structured Streaming	Joins streams, computes congestion index, flags anomalies
Reference data store	PostgreSQL	Static routes, stops, corridors, districts
Metrics store	Cassandra	Time-series congestion and weather metrics, high write throughput
Search / alerts log	Elasticsearch	Searchable alert and event history (Tier 3)
Alert fan-out	Redis pub/sub	Distributes detected anomalies to the alert consumer
Notifications	Discord Webhook	Posts formatted real-time congestion alerts to a channel
Orchestration	Apache Airflow	Daily reference data refresh, baseline recalculation, data quality checks
Dashboard	Streamlit / Grafana	Live congestion map, time-series charts, alert feed
Containerization	Docker Compose	Runs the entire stack together for development and demo
Data sources	Transport for Cairo (GTFS), Google Routes API / OSRM, Open-Meteo	Transit routes, live traffic, weather

7. Demo Narrative for Presentation Day

- Open with the problem: show a map of Cairo and ask "how do you know if your commute will be bad right now?" — then say nobody can answer that today.
- Show the architecture diagram briefly (Kafka -> Spark -> storage -> dashboard -> Discord).

- Show the live dashboard: point at the congestion heatmap, then the time-series chart correlating a rain event with a congestion spike.
- Trigger (or wait for) a live anomaly and show the Discord alert arriving in real time — this is the moment that lands with judges.
- Close with the roadmap: "with real GTFS integration and a production traffic feed, this becomes a real tool for commuters, delivery companies, or city planners."